CAMPUS CAFÉ ORDERING SYSTEM

Java Object-Oriented Programming Laboratory Exercise
Written Report

Name: Kurt Andrey Ardosa
Course / Section: BSIT — ITS 202 (Object-Oriented Programming)
Date: November 9, 2026
Exercise Title: Campus Café Ordering System

1.Concept Explanation

1. if Statement: Used in Pastry.computePrice() to check if a pastry is warmed and add an extra fee, and in Meal.computePrice() to add the upsize fee.

2. if-else Statement: Used in MenuItem.setBasePrice() for input validation, in Beverage.computePrice() for size-based pricing, in Order.addItem() to enforce the three-item limit, and in Order.computeTotal() to apply student discounts.

3. Classes and Objects: Applied through seven distinct classes (MenuItem, Beverage, Pastry, Meal, Customer, Order, and CafeApp), with instances instantiated inside CafeApp.main().

4. Attributes and Behaviors: Utilized across every class via private/protected instance variables (state) and instance methods (behaviors) to model real-world café entities.

5. Object Interaction: Implemented in the Order class, which holds references to a Customer and up to three MenuItem objects, allowing methods like printReceipt() to update customer loyalty points.

6. Java Naming Conventions: Applied strictly throughout the project, following PascalCase for class names, camelCase for methods and variables, and descriptive naming standards.

7. Default Constructor: Provided across MenuItem, Beverage, Pastry, Meal, and Customer to initialize objects with default fallback values (e.g., "Unnamed Item", "S" size).

8. Parameterized Constructor: Implemented across subclasses, Customer, and Order to initialize objects with custom attributes upon creation.

9. Constructor Overloading: Applied in the Customer class by providing three distinct constructors (Customer(), Customer(String), and Customer(String, boolean)).

10. public, private, and protected Access Modifiers: Used private for strict encapsulation of fields, protected for shared attributes in MenuItem, and public for accessible methods.

11. Getters, Setters, and Data Hiding: Implemented validated setters (such as rejecting negative prices in setBasePrice) to protect internal object data from invalid states.

12. Method Overloading: Implemented in Customer.addPoints(int) vs. addPoints(double) and Order.computeTotal() vs. Order.computeTotal(double) to handle different parameter types.

13. Method Overriding: Used in subclasses (Beverage, Pastry, Meal) to override computePrice() and getDescription() using the @Override annotation for polymorphic behavior.

14. Abstraction: Applied by declaring MenuItem as an abstract class containing the abstract method computePrice(), forcing all subclasses to provide their own pricing logic.

2. Program Actual Output

Console output generated by running CafeApp.main() (replace with actual terminal screenshot if required by instructor):

```
Order #1001 — Ana Reyes (Student)
Iced Latte [B-01] Size: L ..... 120.00
Ensaymada [P-01] (Warmed) ..... 55.00
Chicken Adobo [M-01] Upsized, 1 side . 142.00
Order is full. Cannot add Cookie [P-02].
Subtotal: 317.00
Student discount (10%): -31.70
TOTAL: 285.30
Loyalty points: 5
Error: base price cannot be negative.

Order #1002 — Ben Cruz
Hot Chocolate [B-02] Size: M ..... 95.00
Beef Tapa [M-02] Regular, 0 sides .... 130.00
Subtotal: 225.00
Voucher: -20.00
TOTAL: 205.00
Loyalty points: 4
```

3. Challenges Encountered and Solutions

• Managing Multiple Items Without Collections

Challenge: The restriction against using ArrayList or any java.util collections meant items could not be stored in a dynamic list.

Solution: Used individual instance variables (item1, item2, item3) inside the Order class alongside an itemCount tracker. The addItem() method checks slot availability sequentially before storing the item.

• Implementing Polymorphic Subtotal Calculations

Challenge: Summing prices across different item types without knowing their exact subclass at compile time.

Solution: Leveraged polymorphism by calling computePrice() directly on each MenuItem reference stored in the order, allowing Java's dynamic method dispatch to execute the correct subclass implementation.

• Encapsulation and Data Validation

Challenge: Ensuring that negative prices or invalid beverage sizes not equal to "S", "M", or "L" , would not corrupt object states.

Solution: Implemented strict validation checks inside setter methods using if-else conditionals that reject invalid input and output informative error messages while retaining safe default values.